

Curso de Go

Aula 21 - Conectando Go a Bancos de Dados

Professor : Antônio Marcelo

NÚCLEA
ACADEMY

 **código_** [3]
BRAZUCA





Nessa Aula nos Vamos Ver

- **O pacote Database**
- **O PostgreSQL**
- **Abertura de Conexões**
- **Testando Conexão**
- **Criando uma tabela**
- **Inserindo Dados**
- **Consultando um registro**
- **Consultando Vários Registros**
- **UPDATE e DELETE**
- **Boas Práticas**
- **Parte Prática**



Conectando Go a Bancos de Dados

- **Aplicações reais normalmente precisam armazenar informações de maneira permanente.**
- **Exemplos:**
 - **Usuários**
 - **Produtos**
 - **Pedidos**
 - **Configurações**
 - **Logs**
 - **Dados financeiros**
- **Em Go, uma das principais formas de trabalhar com bancos relacionais é utilizando o pacote:**
 - **database/sql**



O pacote database/sql

- O pacote database/sql faz parte da biblioteca padrão do Go.

- Importação:

```
import "database/sql"
```

- Ele permite:
- Abrir conexões
- Executar comandos SQL
- Realizar consultas
- Trabalhar com transações
- Utilizar prepared statements
- Gerenciar pool de conexões
- Porém, database/sql não implementa diretamente a comunicação com PostgreSQL, MySQL ou outros bancos. Para isso precisamos de um driver.



PostgreSQL

- PostgreSQL é um sistema gerenciador de banco de dados relacional de código aberto.
- É muito utilizado em:
- APIs
- Sistemas Web
- Aplicações corporativas
- Microserviços
- Sistemas financeiros
- Aplicações em nuvem
- Exemplo de banco: loja / tabela Produtos

```
produtos
id | nome      | preco
1  | Notebook | 3500
2  | Mouse    | 80
```




Instalando o Driver PostgreSQL

- Uma opção moderna para trabalhar com PostgreSQL em Go é o driver **pgx**.
- **Instalação:**

```
go get github.com/jackc/pgx/v5/stdlib
```

No Código

```
import (  
    "database/sql"  
  
    _ "github.com/jackc/pgx/v5/stdlib"  
)
```

O “_” indica que o pacote está sendo importado para registrar o driver, sem utilizarmos diretamente seus elementos. Depois podemos utilizar:

```
sql.Open("pgx", ...)
```

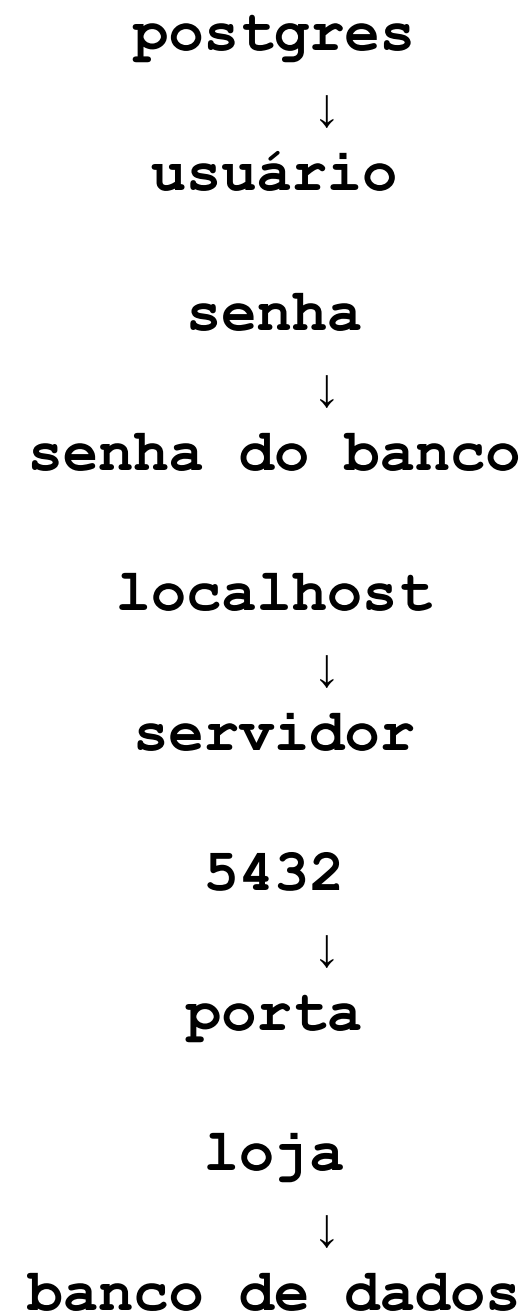


String de Conexão

- Para conectar ao PostgreSQL precisamos informar os dados do servidor. Exemplo:

```
postgres://postgres:senha@localhost:5432/loja
```

- Componentes





Abrindo uma Conexão

- Exemplo básico:

```
package main

import (
    "database/sql"
    "fmt"

    _ "github.com/jackc/pgx/v5/stdlib"
)

func main() {

    db, err := sql.Open(
        "pgx",
        "postgres://postgres:senha@localhost:5432/loja",
    )

    if err != nil {
        panic(err)
    }

    defer db.Close()

    fmt.Println("Banco configurado!")
}
```



A variável db será utilizada para acessar o banco.



Testando a Conexão

- **sql.Open()** não necessariamente estabelece uma conexão imediatamente.
- Podemos testar o acesso utilizando:

```
err := db.Ping()

if err != nil {
    panic(err)
}

fmt.Println("Conectado ao PostgreSQL!")
```

Exemplo completo

```
db, err := sql.Open("pgx", conexao)

if err != nil {
    panic(err)
}

defer db.Close()

if err := db.Ping(); err != nil {
    panic(err)
}
```

O Ping() verifica se o banco realmente está acessível.



Criando uma Tabela

- **Exemplo**

```
sqlTabela := `  
CREATE TABLE IF NOT EXISTS produtos (  
    id SERIAL PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    preco NUMERIC(10,2) NOT NULL  
);  
`
```

```
_ , err := db.Exec(sqlTabela)
```

```
if err != nil {  
    panic(err)  
}
```



Exec() é utilizado para comandos que normalmente não precisam retornar registros. Exemplos:

- **CREATE**
- **INSERT**
- **UPDATE**
- **DELETE**
- **ALTER**



INSERT: Inserindo Dados

- Para inserir um produto:

```
_, err := db.Exec(  
    "INSERT INTO produtos (nome, preco)  
VALUES ($1, $2)",  
    "Notebook",  
    3500.00,  
)  
  
if err != nil {  
    panic(err)  
}
```



No PostgreSQL utilizamos, como parâmetros

- \$1
- \$2
- \$3

Isso é preferível a concatenar valores diretamente no SQL e ajuda a evitar vulnerabilidades como SQL Injection.



INSERT Retornando o ID

- Muitas vezes queremos saber o ID criado pelo PostgreSQL. Podemos utilizar:

```
var id int

err := db.QueryRow(
    `INSERT INTO produtos (nome, preco)
    VALUES ($1, $2)
    RETURNING id`,
    "Teclado",
    150.00,
).Scan(&id)

if err != nil {
    panic(err)
}

fmt.Println("Produto criado:", id)
```

O PostgreSQL cria o ID e o retorna para nossa aplicação Go.



SELECT: Consultando um Registro

- Para buscar somente um registro podemos utilizar: `QueryRow()`.
Exemplo:

```
var nome string
var preco float64

err := db.QueryRow(
    "SELECT nome, preco FROM produtos WHERE
id = $1",
    1,
).Scan(&nome, &preco)

if err != nil {
    panic(err)
}

fmt.Println(nome, preco)
```

O método `Scan()` transfere os dados retornados pelo banco para variáveis Go.



Consultando Vários Registros

- Para retornar vários registros utilizamos: `Query()`. Exemplo:

```
rows, err := db.Query(  
    "SELECT id, nome, preco FROM produtos",  
)  
  
if err != nil {  
    panic(err)  
}  
  
defer rows.Close()  
  
for rows.Next() {  
  
    var id int  
    var nome string  
    var preco float64  
  
    rows.Scan(&id, &nome, &preco)  
  
    fmt.Println(id, nome, preco)  
}
```

O `rows.Next()` percorre os registros retornados pela consulta.



UPDATE e DELETE

- **Atualizando um produto:**

```
_ , err := db.Exec(  
    "UPDATE produtos SET preco = $1 WHERE id = $2",  
    4000.00,  
    1,  
)
```

- **Excluindo um produto:**

```
_ , err := db.Exec(  
    "DELETE FROM produtos WHERE id = $1",  
    1,  
)
```



Temos então as quatro operações básicas:

**CREATE → INSERT
READ → SELECT
UPDATE → UPDATE
DELETE → DELETE**

Essas operações formam o conhecido CRUD.



Migrações

- **Migrações permitem controlar alterações na estrutura do banco de dados ao longo do desenvolvimento. Exemplo:**

```
migrations/
```

```
001_create_produtos.sql
```

```
002_add_estoque.sql
```

```
003_create_clientes.sql
```

- **Exemplo de migração:**

```
ALTER TABLE produtos
```

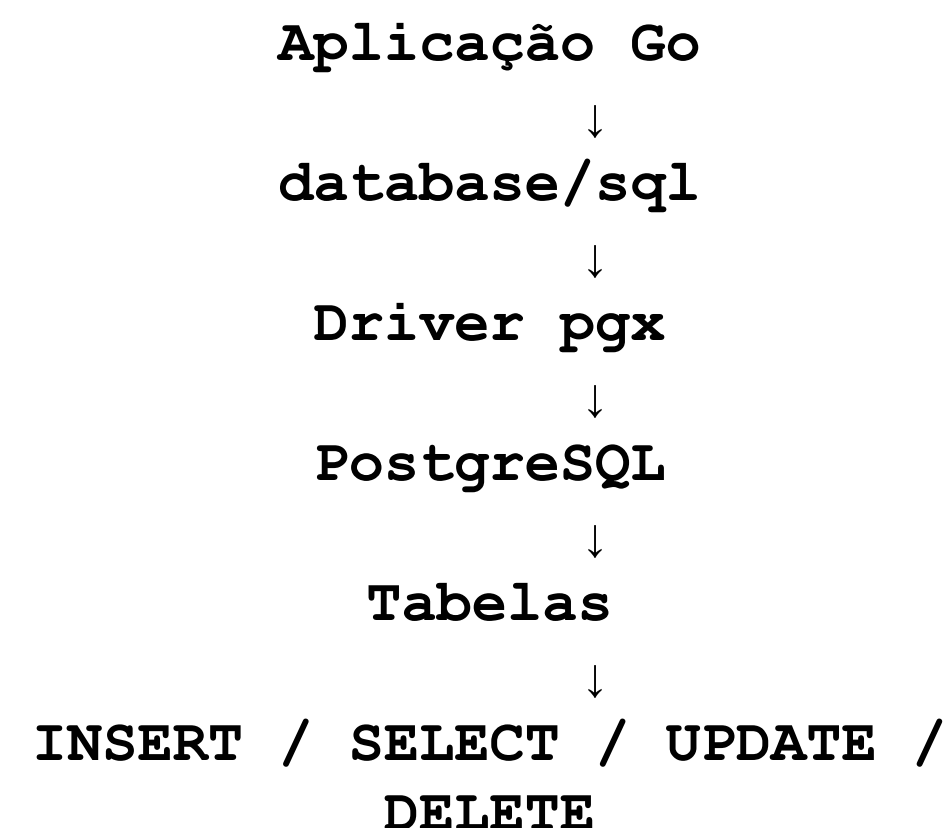
```
ADD COLUMN estoque INTEGER DEFAULT 0;
```

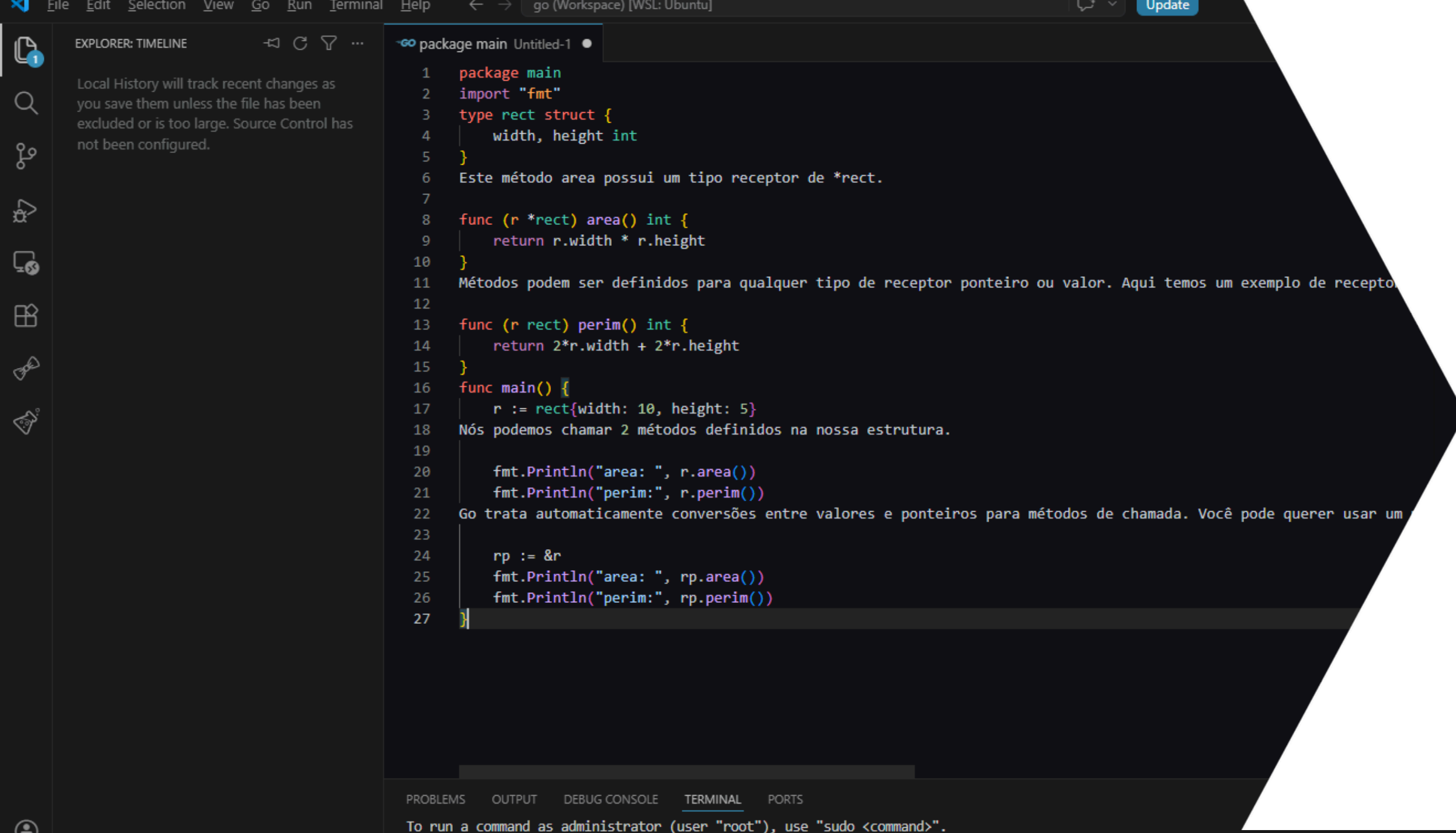


Boas Práticas

- Boas práticas em aplicações Go:
- Utilize database/sql ou bibliotecas compatíveis.
- Utilize parâmetros \$1, \$2, etc.
- Nunca concatene dados do usuário diretamente em queries.
- Utilize defer rows.Close().
- Verifique os erros retornados pelo banco.
- Utilize variáveis de ambiente para senhas.
- Separe código de acesso a dados da lógica da aplicação.
- Utilize migrações para controlar mudanças no banco.
- Utilize transações quando várias operações precisarem ocorrer juntas.
- Configure adequadamente o pool de conexões.

Fluxo Básico





NÚCLEA
ACADEMY

Parte Prática

Conectando Go a Bancos de Dados